PRÁCTICA: Creación de aplicaciones en C (I)

Objetivos:

-Manejo del Lenguaje C

-Uso de funciones de librería y ficheros cabecera

-Etapas y uso del compilador: gcc

-Manejo de ficheros usando la librería estándar de C

-Crear librerías dinámicas y estáticas

-Gestionar los proyectos con make

Sabe instalar lo necesario para programar en c y sus librerías

Instala o comprueba que están instalados los siguientes paquetes o aplicaciones:

Compilador, librerías y documentación necesaria para programar en C:

sudo apt update

sudo apt install build-essential // Con este paquete se instalan los paquetes esenciales para programar en C

Además de ayuda y documentación extra:

sudo apt install debian-keyring //GnuPG keys of Debian Developers,las **claves GnuPG** (GNU Privacy Guard) de los desarrolladores y mantenedores se usan principalmente para **verificar la autenticidad e integridad de los paquetes**.

sudo apt install gcc-doc //Documentation for the GNU compilers (gcc, gobjc, g++)

sudo apt install glibc-doc //Embedded GNU C Library: Documentation

sudo apt install manpages-dev //Manual pages about using GNU/Linux for development

Instala la ayuda y documentación de la APIs de POSIX:

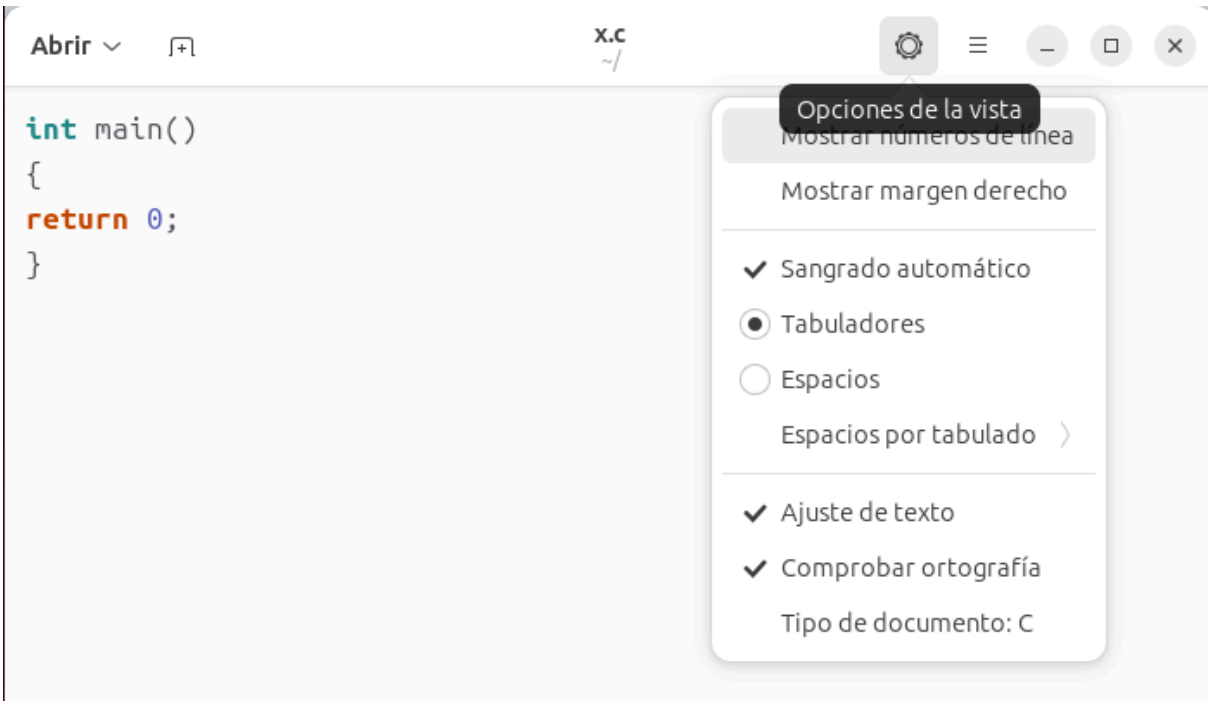
sudo apt install manpages-posix //Manual pages about using POSIX system

POSIX (Portable Operating System Interface, y X viene de UNIX como señal de identidad de la API), norma que define una interfaz estándar del sistema operativo y el entorno, incluyendo un intérprete de comandos (o "shell"), y programas de utilidades comunes para apoyar la portabilidad de las aplicaciones a nivel de código fuente. El nombre POSIX surgió de la recomendación de Richard Stallman, que por aquel entonces en la década de 1980 formaba parte del comité de IEEE.

Sabe usar un editor de texto que reconoce C

Editor de C:

```
$gnome-text-editor
```



Sabe utilizar una VS Code que reconoce C

https://docs.google.com/presentation/d/1P45EnDl08bN4KArHoCHsrW3EreiFZgg6rzrh0JK-Lko/edit?usp=sharing

Sabe manejarse entre los ficheros intermedios de las diferentes etapas de compilación

1.- Crea un programa fuente “pi.c” que contenga el código siguiente:

```
#include <stdio.h>
#define PI 3.1415
int main ()
{
char c;
printf("¿Quieres conocer al número PI? (S/N)");
scanf("%c",&c);
if (c=='S' || c=='s') printf("%f\n",PI);
printf("Fin de programa \n");
return 0;
}
```

a) Preprocesa el programa pi.c y analiza el código.
nachet@nacho:**~/practicas-c/practica1**$ gcc -o pi.i -E pi.c

b) Obtén pi.s (ensamblado).
nachet@nacho:**~/practicas-c/practica1**$ gcc -o pi.s -S pi.c

c) Obtén pi.o (objeto).
nachet@nacho:**~/practicas-c/practica1**$ gcc -o pi.o pi.c

d) Obtén el ejecutable pi a partir del fuente.
nachet@nacho:**~/practicas-c/practica1**$ gcc -o pi pi.c

Sabe usar la ayuda a la programación

Utilización del man:

A partir de ahora vamos a utilizar comandos del "*bash*", funciones de librería de C también conocidas como las "API-s de C", funciones de librería del SO Linux también conocidas como las "API-s de Linux". Para obtener la ayuda correcta dentro del man, vamos a analizar las secciones de la ayuda del man:

```
$man man
NOMBRE
      man - interfaz de los manuales de referencia del sistema

SINOPSIS
      man [opciones de man] [[sección] página ...] ...
      man -k [opciones de apropos] regexp ...
      man -K [opciones de man] [sección] term ...
      man -f [whatis opciones] página ...
      man -l [opciones de man] archivo ...
      man -w|-W [opciones de man] página ...

DESCRIPCIÓN
      man  es  el  paginador de manuales del sistema.  Cada argumento de página dado a
man normalmente es el nombre de un programa, utilidad o función. La página de manual
asociada
      con cada uno de estos argumentos es, pues, encontrada y mostrada.  Si se proporciona
una sección, man mirará solo en esa sección del  manual.  La  acción  predeterminada  es
      buscar  en todas las secciones disponibles siguiendo un orden predefinido (véase
DEFAULTS), y mostrar solo la primera página encontrada, incluso si la página existe en
varias
      secciones.

      La tabla de abajo muestra los números de sección del manual seguidos por los tipos de
página que contienen.
Nº de Sección - Tema
```

1 Programas ejecutables u órdenes de la shell
2 Llamadas al sistema (funciones proporcionadas por el núcleo)
3 Llamadas a biblioteca (funciones dentro de bibliotecas de programa)
4 Archivos especiales (normalmente se encuentran en /dev)
5 Formatos de archivo y convenios, p.e. /etc/passwd
6 Juegos
7 Miscelánea (incluidos paquetes de macros y convenios), p.e. man(7), groff(7), man-pages(7)
8 Órdenes de administración del sistema (normalmente solo para root)
9 Rutinas del núcleo [No estándar]

EJEMPLOS
$man ls, Enseña la página de manual para el ítem (programa) ls.

$ man 1 write, Busca "write" en la sección 1 del manual.

$ man 2 write, Busca "write" en la sección 2 del manual.
$ man 5 shadow, Busca "shadow" en la sección 5 del manual.
$ man -a passwd, Busca "passwd" en todas las secciones del manual. Saltando de sección en sección. Aparece información de todas(-a de "all") las secciones

Sabe dónde están los ficheros cabeceras y librerías en linux

1.- ¿Qué contiene el fichero stdio.h?
whereis stdio.h
stdio.h: /usr/include/stdio.h
kepa@kepa-Precision-3551:~$ gnome-text-editor /usr/include/stdio.h
cabeceras de las funciones de entrada y salida como printf
2.- Localiza el prototipo/declaración/cabecera de la función printf en /usr/include/stdio.h
extern int printf (const char *__restrict __format, ...);
3.- ¿Dónde están las librerías o bibliotecas libc.so y libc.a estándar de C en linux?
whereis libc.a
libc.a: /usr/lib/x86_64-linux-gnu/libc.a
whereis libc.so
libc.so: /usr/lib/x86_64-linux-gnu/libc.so
4.- Para más información échale un vistazo a la documentación de la librería estándar de C.
http://www.gnu.org/software/libc/manual/html_mono/libc.html

Todo en el main

Sabe abrir un fichero en modo lectura

1.- Realiza un programa en C que preguntando al usuario el nombre de un fichero calcule su tamaño en bytes utilizando las funciones **fopen, fgetc y fclose** de las librerías de C . Compila y ejecuta el ejecutable pasándole un fichero de texto, y comprueba que coincide con la salida del comando "ls -l" como se indica en el ejemplo:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
   char nombre[100];
   int count =0;
   printf("¿De qué fichero deseas conocer el tamaño?");
   scanf("%s", nombre);

   FILE *file = fopen(nombre, "r");
   if (file == NULL) {
       printf("No se pudo abrir el archivo.\n");
       return 1;
   }
   while(fgetc(file) != EOF){
       count++;
   }
   fclose(file);
   printf("El tamaño del fichero es: %d\n", count);
   return 0;
}
```

```
nachet@nacho:~/practicas-c/practica1$ gcc -o longFich longFich.c
nachet@nacho:~/practicas-c/practica1$ ./longFich
¿De qué fichero deseas conocer el tamaño?longFich.c
El tamaño del fichero es: 454
nachet@nacho:~/practicas-c/practica1$ ls -l longFich.c
-rw-rw-r-- 1 nachet nachet 454 mar 23 15:47 longFich.c
nachet@nacho:~/practicas-c/practica1$
```

Sabe abrir un fichero en modo escritura

1.- Realiza un programa en C que preguntando al usuario por el nombre de un fichero escriba en este el abecedario tanto en minúsculas como en mayúsculas, cada uno en una fila, utilizando las funciones **fopen, fputc y fclose** de las librerías de C .Compila y ejecuta el ejecutable, y comprueba con **less** que la salida es como se indica en el ejemplo:

```
#include <stdio.h>
int main() {
   char nombre[100];
   int count =0;
   printf("Introduce el nombre del fichero a crear:");
   scanf("%s", nombre);

   FILE *file = fopen(nombre, "w");
   if (file == NULL) {
       printf("No se pudo abrir el archivo.\n");
       return 1;
   }
   for (char c='a'; c<='z'; c++) {
       fputc(c, file);
       count++;
   }
   fputc('\n', file);
   for (char c='A'; c<='Z'; c++) {
       fputc(c, file);
       count++;
   }
   fputc('\n', file);
   fclose(file);
   return 0;
}
```

nachet@nacho:~/practicas-c/practica1$ gcc -o abece abece.c
nachet@nacho:~/practicas-c/practica1$ ./abece
Introduce el nombre del fichero a crear:prueba.txt
nachet@nacho:~/practicas-c/practica1$ less prueba.txt

Sabe utilizar los ejecutables en otros programas:

1.-Ejecuta el siguiente programa por lotes o script de Bash que ejecute los dos ejecutables realizados en los ejercicios anteriores. Si el primer ejecutable se ejecuta correctamente que se ejecute el segundo. Para ello si el ejecutable “abece” termina correctamente, ejecute “longFich”, y si no muestre un mensaje de que el primer programa ha fallado. El programa se detalla abajo. Comprueba su uso correcto.

$gnome-text-editor **bashscript.sh**

```
#!/usr/bin/bash
./abece
if [ $? -eq 0 ] ; then
        ./longFich
        if [ $? -eq 0 ] ; then
            echo -e "\nwell done"
            exit 0
        else
            echo -e "\nfailed"
            exit 1
        fi
else
        echo -e "\nfailed"
        exit 1
fi
```

$chmod u+x ./bashscript.sh

$./bashscript.sh

main+módulos

Sabe gestionar ficheros usando librerías propias

1.-El siguiente programa principal “numeroaes.c” está incompleto:

```
#include <stdio.h>
int ffichnumcarac(char c, FILE * fp);
int main()
{
char nombre[80],c;
FILE *fp;
int contador=0;
printf ("¿De qué fichero deseas conocer el número de aes?");
scanf("%s",nombre);
```

```
fp=fopen(nombre,"r");
if (fp==NULL)
      {
          printf("No se puede abrir el fichero %s \n",nombre);
          return -1;
       }
else
      {      contador=ffichnumcarac('a',fp);
             fclose(fp);
             printf("El número de a-s es de %d",contador);
      }
return 0;
}
int ffichnumcarac(char c, FILE * fp)
{


}
```

Se pide:

a)Completa la función “ffichnumcarac” dentro de “numeroaes.c”. A esta función se le pasarán como parámetros el carácter a buscar y el descriptor de fichero. Y nos devolverá el número de veces que aparece dicho carácter en el fichero. Compila y ejecuta dicho programa principal *numeroaes*.

```
#include <stdio.h>
int ffichnumcarac(char c, FILE * fp);
int main() {
   char nombre[80],c;
   FILE *fp;
   int contador=0;
   printf ("¿De qué fichero deseas conocer el número de aes?");
   scanf("%s",nombre);
   fp=fopen(nombre,"r");
```

```
        if (fp==NULL)
                {
                printf("No se puede abrir el fichero %s \n",nombre);
                return -1;
                }
        else
                {       contador=ffichnumcarac('a',fp);
                        fclose(fp);
                        printf("El número de a-s es de %d \n",contador);
                        return contador;
                }
}
int ffichnumcarac(char c, FILE * fp)
{
        int cont=0;
        char caracter;
        while ((caracter=fgetc(fp))!=EOF) {
                if (caracter==c)
                        cont++;
        }
        return cont;
}
```

nachet@nacho:**~/practicas-c/practica1**$ gcc -o numeroaes numeroaes.c

nachet@nacho:**~/practicas-c/practica1**$ ./numeroaes

¿De qué fichero deseas conocer el número de aes?numeroaes.c

El número de a-s es de 27

b)Ahora introduce dicha función “ffichnumcarac” en un módulo que se llamará “general.c”. Compila el módulo general y el módulo principal(main) numeroaes por separado y finalmente crea el ejecutable numeroaes con dichos módulos objeto.

```
#include <stdio.h>

#include "general.h"

int main() {

   char nombre[80],c;

   FILE *fp;

   int contador=0;

   printf ("¿De qué fichero deseas conocer el número de aes?");

   scanf("%s",nombre);

   fp=fopen(nombre,"r");

       if (fp==NULL)

               {

               printf("No se puede abrir el fichero %s \n",nombre);

               return -1;

               }

       else

               {       contador=ffichnumcarac('a',fp);

                       fclose(fp);

                       printf("El número de a-s es de %d \n",contador);

                       return contador;

               }

}
```

```c
#include "general.h"

int ffichnumcarac(char c, FILE * fp)

{

	int cont=0;

	char caracter;

	while ((caracter=fgetc(fp))!=EOF) {

		if (caracter==c)

			cont++;

	}

	return cont;

}
```

```c
#ifndef GENERAL_H

#define GENERAL_H

#include <stdio.h>

int ffichnumcarac(char c, FILE * fp);

#endif
```

nachet@nacho:**~/practicas-c/practica1**$ gcc -c general.c -o general.o

nachet@nacho:**~/practicas-c/practica1**$ gcc -c numeroaes.c -o numeroaes.o

nachet@nacho:**~/practicas-c/practica1**$ gcc -o numeroaes general.o numeroaes.o

librerías estáticas y dinámicas

b) Ahora introduce dicha función “ffichnumcarac” en un módulo que se llamará “general.c”. Finalmente introduce dicho módulo en una biblioteca estática cuyo nombre será “libgeneral.a”. Y crea el ejecutable *numeroaes2* usando dicha librería estática. Para ello modifica el programa principal para que la compilación tenga éxito.

nachet@nacho:**~/practicas-c/practica1**$ gcc -c general.c -o general.o

nachet@nacho:**~/practicas-c/practica1**$ ar rv libgeneral.a general.o

ar: creando libgeneral.a

a - general.o

nachet@nacho:**~/practicas-c/practica1**$ ranlib libgeneral.a

nachet@nacho:**~/practicas-c/practica1**$ nm -s libgeneral.a

nachet@nacho:**~/practicas-c/practica1**$ gcc -L. -I. -o numeroaes2 numeroaes.c -lgeneral -static

nachet@nacho:**~/practicas-c/practica1**$ ./numeroaes2

c) Ahora introduce dicha función “ffichnumcarac” en un módulo que se llamará “general.c”. Finalmente introduce dicho módulo en una biblioteca dinámica cuyo nombre será “libgeneral.so”. Y crea el ejecutable *numeroaes* usando dicha librería dinámica. Para ello modifica el programa principal para que la compilación tenga éxito.

nachet@nacho:~/practicas-c/practica1$ gcc -c general.c -o general.o -fPIC
nachet@nacho:~/practicas-c/practica1$ ld -o libgeneral.so general.o -shared
nachet@nacho:~/practicas-c/practica1$ gcc -o numeroaes3 numeroaes.c -L. -I. -lgeneral -ByDinamic
nachet@nacho:~/practicas-c/practica1$ ./numeroaes3
./numeroaes3: error while loading shared libraries: libgeneral.so: cannot open shared object file: No such file or directory

nachet@nacho:~/practicas-c/practica1$
LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/home/nachet/practicas-c/practica1
nachet@nacho:~/practicas-c/practica1$ export LD_LIBRARY_PATH
nachet@nacho:~/practicas-c/practica1$ gcc -o numeroaes3 numeroaes.c -L. -I. -lgeneral -ByDinamic
nachet@nacho:~/practicas-c/practica1$ ./numeroaes3
¿De qué fichero deseas conocer el número de aes?numeroaes.c
El número de a-s es de 15
nachet@nacho:~/practicas-c/practica1$

Sabe ejecutar un comando de bash shell desde C

¿Para qué sirve la función system?. Ejecuta el siguiente código:

```
#include<stdio.h>

#include <stdlib.h>
int main(){
system("ls -l");
return 0;
}
```

```
nachet@nacho:~/practicas-c/practica1$ gcc -o main main.c

nachet@nacho:~/practicas-c/practica1$ ./main

total 1144

-rwxrwxr-x 1 nachet nachet  16232 mar 23 15:55 abece

-rw-rw-r-- 1 nachet nachet    534 mar 23 15:55 abece.c

-rw-rw-r-- 1 nachet nachet    244 mar 24 20:20 general.c

-rw-rw-r-- 1 nachet nachet    223 mar 24 20:12 general.c.c

-rw-rw-r-- 1 nachet nachet    100 mar 24 20:20 general.h

-rw-rw-r-- 1 nachet nachet   1416 mar 24 20:50 general.o

-rw-rw-r-- 1 nachet nachet   1566 mar 24 20:29 libgeneral.a

-rwxrwxr-x 1 nachet nachet  13800 mar 24 20:50 libgeneral.so

-rwxrwxr-x 1 nachet nachet  16232 mar 23 15:47 longFich

-rw-rw-r-- 1 nachet nachet    454 mar 23 15:47 longFich.c

-rwxrwxr-x 1 nachet nachet  15960 mar 24 21:01 main

-rw-rw-r-- 1 nachet nachet     87 mar 24 21:01 main.c

-rwxrwxr-x 1 nachet nachet  16264 mar 24 20:21 numeroaes

-rwxrwxr-x 1 nachet nachet 915984 mar 24 20:40 numeroaes2

-rwxrwxr-x 1 nachet nachet  16192 mar 24 20:59 numeroaes3

-rw-rw-r-- 1 nachet nachet    625 mar 24 20:19 numeroaes.c

-rw-rw-r-- 1 nachet nachet   2264 mar 24 20:20 numeroaes.o

-rwxrwxr-x 1 nachet nachet  33456 mar 23 14:41 pi

-rw-rw-r-- 1 nachet nachet    232 mar 23 14:41 pi.c

-rw-rw-r-- 1 nachet nachet  25921 mar 23 14:41 pi.i

-rwxrwxr-x 1 nachet nachet  33464 mar 23 14:41 pi.o

-rw-rw-r-- 1 nachet nachet   1645 mar 23 14:41 pi.s

-rw-rw-r-- 1 nachet nachet     54 mar 23 15:55 prueba.txt
```

El comando system() sirve para encadenar comandos del sistema.